

- The Problem

- Making test cases and suites can be hard especially for small teams. Writing up test suites sometimes takes more time than actually writing the implementation its trying to test, this can be time consuming and frustrating for developers, talking from experience here. This combined with tight deadlines makes the scenario a thousand times worse. So developers end up making bad test cases and rushing the process to get the product out, sometimes even skips writing suites. So the task is to make a program/application that makes testing less manual and try and automate finding bugs in the code you produce.

- My Approach

- After initially reading the brief and the requirements, multiple ideas came to mind. URL web crawler, app where you can directly upload source code, web app where it scans through the github repo and a extension that you can install on your IDE directly. After thinking more about how i would make it it came down to repo scanning and file/folder uploading. URL web crawlers only works with UI rendered on the browser and doesnt look at the code it self and building a web crawler that works for any app within a 2 week timeline requires a lot of time and i personally didnt think i could finish it before the deadline. And the extension idea wouldve been great but i have never made a extension before and with the deadline not being that far away learning new techniques and concepts would take too much time. So repo scanning and uploading folders were my initial idea for the app.
- I wanted make a web app where a user can paste a repo link or upload a zip folder and it would scan through them and pass it on to an ai(LLM) and look for edge cases, bugs, maintenance issues.

- Out of Scope

- **UI testing:** since the app was looking at directly the source code it meant that it wouldnt support UI testing by default. Testing the UI requires stepping through each stage or pages of an app and trying to essentially break it. Trying to change the viewbox to see if the UI is dynamic and adaptive, which is what a web crawler would do, but as ive mentioned before this is complex for a tight deadline and making a demo work this quick can open up situations for high runtime risks and a lot of failure points.
- **Folder uploads:** i initially wanted to have folder uploads in the web app, but after half way through making the web app i decided to drop that part of the app. I felt like it added more work instead of making it more automated. Users have compress the whole project and upload it to the app then either i can extract on the frontend and send to the back which might cause some performance issues or i can send the whole zip file to the back ut then it would need server-side storage for each user.